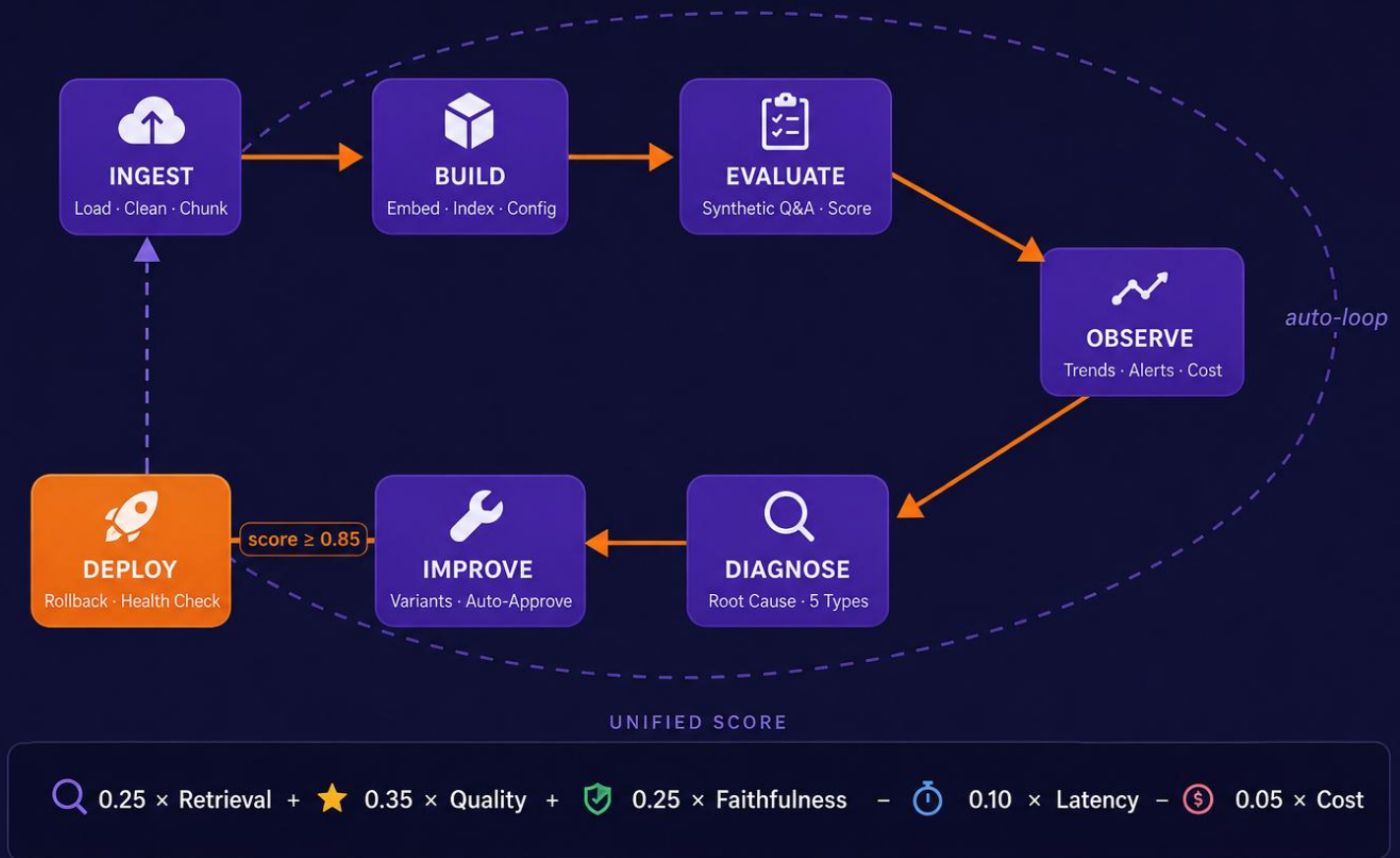


# AutoRAG

Self-Improving RAG Platform with Autonomous Knowledge Compression

## Product Requirements Document



🔄 CLOSED-LOOP · 🔄 SELF-IMPROVING · ⚙️ NO MANUAL TUNING

Version 1.0 | May 2026 |

Euron AI Product Engineering 2.0 | For Review & Distribution

Table of Contents

01. Executive Summary.....2

02. Problem Statement.....3

03. Target Users & Personas.....4

04. Product Goals & Success Metrics.....5

05. Scope Definition — MVP.....8

06. System Architecture.....9

07. Agentic AI Design.....10

08. Feature Breakdown.....13

09. Observer Agent — Monitoring & Observability.....17

10. Rollback Strategy.....18

11. Improvement Approval Framework.....19

12. Pipeline Configurator — Feature Spec.....20

13. API & Integration Requirements.....21

14. Data & Storage Design.....22

15. Non-Functional Requirements.....23

16. AI Safety & Governance.....24

17. Testing Strategy.....25

18. Deployment & DevOps.....26

19. UX & UI Requirements.....27

20. Timeline & Milestones.....28

21. Business Model & Pricing.....29

22. Risks & Assumptions.....30

23. Open Questions & Sign-Off.....31

## 01 Executive Summary

Vision: Eliminate the manual iteration cycle in RAG pipeline development with a fully autonomous, closed-loop system that builds, evaluates, observes, diagnoses, improves, and deploys RAG pipelines — **without requiring labeled data.**

### 1.1 Product Name & Category

Product: AutoRAG | Category: Autonomous AI Quality Systems | Version: 1.0 | May 2026

### 1.2 7-Agent Architecture

| Agent     | Technical Role                                                                 | Priority |
|-----------|--------------------------------------------------------------------------------|----------|
| Ingestion | Document loading, cleaning, chunking — PDF/HTML/TXT → MinIO + PostgreSQL       | P0       |
| Builder   | Embedding pipeline construction + Qdrant vector indexing from YAML config      | P0       |
| Evaluator | Synthetic Q&A generation + LLM-as-judge scoring across 4 dimensions            | P0       |
| Observer  | Score trend monitoring, query success rate, failure breakdown, cost vs quality | P0       |
| Diagnoser | Failure classification (5 types) + root-cause analysis + recommendations       | P0       |
| Improver  | Experiment orchestration — generates variants, tests all, selects best         | P0       |
| Deployer  | Docker/K8s deployment with health checks, rollback, and versioning             | P1       |

### 1.3 Unified Scoring Formula

$$\text{Score} = 0.25 \times \text{Retrieval} + 0.35 \times \text{Quality} + 0.25 \times \text{Faithfulness} - 0.10 \times \text{Latency} - 0.05 \times \text{Cost}$$

### 1.4 Business Impact

| Dimension         | Before AutoRAG                    | With AutoRAG                   |
|-------------------|-----------------------------------|--------------------------------|
| Iteration Cycle   | 2–3 engineering days              | < 15 minutes (projected)       |
| Cost per Pipeline | \$3,000–\$8,000+ engineering time | < \$500 compute + API          |
| Data Labeling     | Weeks of expert annotation        | Zero — synthetic evaluation    |
| Quality Gate      | Ad-hoc, inconsistent              | ≥ 0.80 Unified Score every run |

## 02 Problem Statement

### 2.1 The Problem

Building a production-quality RAG system requires manual, iterative tuning across 5 independent layers with no systematic feedback between them.

| Layer                   | The Challenge                                                                           |
|-------------------------|-----------------------------------------------------------------------------------------|
| Chunking strategy       | Fixed-size vs semantic vs hierarchical — each changes retrieval behaviour unpredictably |
| Embedding model         | Quality vs cost trade-off with no reliable benchmark for a given domain                 |
| Retrieval configuration | Dense vs sparse vs hybrid, top-k — interdependent with chunking decisions               |
| Prompt engineering      | System prompt and query formulation — no automated way to test variants                 |
| Generation settings     | Temperature, context window, model — directly affects faithfulness and cost             |

### 2.2 Current Solutions & Limitations

| Approach                           | Limitation                                                     |
|------------------------------------|----------------------------------------------------------------|
| Manual testing with sample queries | Not systematic; no ground truth; not reproducible across teams |
| Ragas standalone                   | Evaluation only — no pipeline builder, no improvement loop     |
| LangSmith / LangFuse               | Tracing and logging only — no autonomous improvement           |
| Custom eval scripts                | One-off; not generalizable; requires labeled data              |
| LlamaIndex pipelines               | Build only — no evaluation or improvement feedback             |

### 2.3 Why Now

- ▶ LLM API costs dropped **10× in 18 months** — autonomous multi-agent orchestration is now economically viable
- ▶ Enterprise internal knowledge base deployments are **now mainstream** — the bottleneck has shifted from 'can we build it' to 'how do we know it is good'
- ▶ Synthetic evaluation using LLMs-as-judges is **now reliable enough to replace human labeling** for most RAG use cases

## 03 Target Users & Personas

### 3.1 AI/ML Engineer

ML Engineer at a mid-size SaaS company. Builds and maintains 2–3 RAG pipelines for different products. Uses desktop + terminal daily.

- Goals: **Build a production RAG pipeline and evaluate it within one working session; compare multiple configs side-by-side with reproducible, versioned results**
- Pain Points: **No systematic evaluation framework; relies on ad-hoc spot-checking; manual iteration with no automated feedback loop**
- Configurator use: Uses Configurator output as a starting point — edits the full YAML directly in code for fine-tuned control

### 3.2 Data Scientist

Data Scientist at a research-oriented team. Runs experiments on retrieval strategies for document-heavy domains. Uses Jupyter notebooks.

- Goals: **Run controlled, reproducible experiments; measure answer quality objectively with a consistent scoring framework**
- Pain Points: **No ground truth dataset; labeling is expensive; no unified scoring that accounts for retrieval, quality, and cost together**
- Configurator use: Bootstraps a domain-tuned experiment baseline in minutes; adjusts scoring weights manually from there

### 3.3 Enterprise Team Lead

Platform Team Lead at a 500-person enterprise. Deploying internal HR/legal knowledge base for employee self-service. Requires on-premise deployment.

- Goals: **Deploy a reliable internal knowledge base; maintain full on-premise data control; no documents leave the company network**
- Pain Points: **No turnkey deployment automation; no monitoring or rollback when production quality degrades**
- Configurator use: Generates a governance-first config — human approval mode pre-selected, strict faithfulness weights set — before handing off to engineers

### 3.4 Solo Developer

Solo developer building a RAG-powered SaaS for legal professionals. Needs to ship fast and optimise costs.

- Goals: **Working RAG endpoint in under 10 minutes from a single docker-compose command; optimise cost-quality trade-off**
- Pain Points: **Complex multi-component stack; no benchmarking tool to validate quality before shipping to paying customers**
- Configurator use: Answers 5 questions and gets a production-ready config — no YAML knowledge required

## 04 Product Goals & Success Metrics

### 4.1 AI-Specific KPIs

| KPI                             | Target      | Measured By                                            |
|---------------------------------|-------------|--------------------------------------------------------|
| Composite Unified Score         | $\geq 0.80$ | Evaluator Agent — every pipeline run                   |
| Retrieval Recall@5              | $\geq 0.85$ | Evaluator Agent — retrieval score component            |
| Answer Quality (LLM-as-judge)   | $\geq 0.80$ | AI judge model — clarity and completeness per response |
| Faithfulness / Hallucination    | $\geq 0.80$ | AI judge model — grounding in source documents         |
| Pipeline Build Time             | < 5 minutes | Builder Agent — logged with timestamp per run          |
| Evaluation Cycle (50 Q&A)       | < 5 minutes | Evaluator Agent — logged with timestamp per run        |
| Failure Classification Accuracy | $\geq 90\%$ | Diagnoser Agent — validated vs 200 labelled failures   |
| Query Response Time (p95)       | < 3 seconds | API layer — distributed tracing on every query         |

### 4.2 Understanding Each KPI

The following explains what each KPI measures, why it matters, and how it is calculated — for engineering and product teams implementing the system.

#### 4.2.1 Composite Unified Score $\geq 0.80$

**What it measures:** The overall quality grade for a pipeline run — a single number from 0.0 to 1.0 combining retrieval, quality, faithfulness, speed, and cost via the weighted formula.

**Why it matters:** Enables apples-to-apples comparison of any two pipeline configurations at a glance. A score of  $\geq 0.80$  is the production deployment threshold.

**How calculated:**  $0.25 \times \text{Retrieval} + 0.35 \times \text{Quality} + 0.25 \times \text{Faithfulness} - 0.10 \times \text{Latency} - 0.05 \times \text{Cost}$ , normalised to [0.0, 1.0].

#### 4.2.2 Retrieval Recall@5 $\geq 0.85$

**What it measures:** When a test question is asked, the system retrieves the top 5 document chunks. This metric checks how often the chunk containing the correct answer is within those 5. '@5' = top-5 retrieved.

**Why it matters:** Retrieval is the foundation — if the right information isn't retrieved, no downstream step can produce a correct answer. A score of 0.85 means 85% of questions find the answer in the top 5 chunks.

**How calculated:** Chunk overlap between retrieved chunks and the source paragraph from the synthetic Q&A pair.

#### 4.2.3 Answer Quality $\geq 0.80$

**What it measures:** Whether the generated answer is clear, complete, and relevant to the question — as judged by a separate AI model acting as an evaluator.

**Why it matters:** This is the dimension most correlated with user satisfaction. It receives the highest weight (35%) in the Unified Score formula.

**How calculated:** The configured AI model (same provider as the generator, or a dedicated judge) evaluates each answer against the expected answer and source paragraph. Score averaged across all 50 test questions.

#### 4.2.4 Faithfulness / Hallucination Rate $\geq 0.80$

**What it measures:** Whether each generated answer is grounded in the source documents — i.e., contains no invented or fabricated facts.

**Why it matters:** Hallucination is the most damaging failure mode for enterprise AI. A pipeline scoring below 0.50 on faithfulness is automatically classified as a Hallucination failure and blocked from deployment.

**How calculated:** AI judge checks whether each claim in the answer can be traced back to a retrieved chunk. Averaged across 50 questions.

#### 4.2.5 Pipeline Build Time < 5 minutes

**What it measures:** Time from triggering a pipeline build to having a fully indexed, query-ready Qdrant collection.

**Why it matters:** Fast build time is a direct measure of the operational efficiency AutoRAG promises. Slow builds block iteration.

**How calculated:** Timestamp delta: POST /pipelines → Qdrant indexing complete. Logged per run in pipeline\_runs table.

#### 4.2.6 Evaluation Cycle (50 Q&A) < 5 minutes

**What it measures:** Time to generate 50 synthetic test questions, run each through the pipeline, and produce a full Unified Score.

**Why it matters:** Faster evaluation cycles = faster iteration. This is the core loop AutoRAG automates.

**How calculated:** Timestamp delta: POST /evaluate triggered → evaluation report written to PostgreSQL. Logged per evaluation run.

#### 4.2.7 Failure Classification Accuracy ≥ 90%

**What it measures:** How accurately the Diagnoser Agent classifies a failed question into one of 5 failure types (Retrieval Failure, Context Missing, Hallucination, Irrelevant Answer, Partial Answer).

**Why it matters:** Correct classification drives correct improvement actions. A misclassified failure leads the Improver Agent to apply the wrong fix.

**How calculated:** Validated against a held-out test set of 200 manually labelled real-world RAG failures. Accuracy = correct classifications / 200.

#### 4.2.8 Query Response Time (p95) < 3 seconds

**What it measures:** The 95th percentile response time for POST /query — meaning 95% of all user queries must return an answer in under 3 seconds. The remaining 5% may be slower.

**Why it matters:** A standard enterprise SLA for user-facing AI applications. Pipelines that consistently exceed 3s are penalised in the Latency component of the Unified Score.

**How calculated:** OpenTelemetry distributed tracing on every API request. p95 calculated over a rolling 1-hour window. Alert fires if p95 > 5s.

### 4.3 Business Goals

| Goal                         | Target          | Timeframe            |
|------------------------------|-----------------|----------------------|
| First enterprise licenses    | 10 customers    | 6 months post-MVP    |
| Annual contract value        | ≥ ₹15L (~\$18K) | Year 1               |
| GitHub stars (OSS community) | 1,000           | 3 months post-launch |
| OSS to Enterprise conversion | ≥ 5%            | Year 1               |

### 4.4 Expected Efficiency Gains

*\* Projections based on industry benchmarks. To be validated with pilot customer data post-MVP.*

| Dimension               | Before AutoRAG                    | Expected With AutoRAG                    |
|-------------------------|-----------------------------------|------------------------------------------|
| Iteration Cycle         | 2–3 engineering days              | < 15 minutes (projected)                 |
| Cost per Pipeline Build | \$3,000–\$8,000+ engineering time | < \$500 compute and API cost (projected) |
| Data Labeling           | Weeks of expert annotation        | Zero — synthetic evaluation (by design)  |
| Quality Assurance       | Ad-hoc spot-checks                | Automated score ≥ 0.80 every run         |
| Production Rollback     | Hours to diagnose and redeploy    | One-click, under 2 minutes (projected)   |

## 05 Scope Definition — MVP

### 5.1 In Scope — P0 Features

| Feature               | Priority | Description                                                                         |
|-----------------------|----------|-------------------------------------------------------------------------------------|
| Ingestion Layer       | P0       | PDF/HTML/TXT loaders, fixed-size and semantic chunkers, MinIO + PostgreSQL storage  |
| Builder Agent         | P0       | 2 embedding models (OpenAI + MiniLM), Qdrant indexing, YAML config                  |
| RAG Runtime           | P0       | Dense + hybrid retriever, configurable generator, prompt templates                  |
| Evaluator Agent       | P0       | Synthetic Q&A generation, LLM-as-judge, Unified Scoring                             |
| Observer Agent        | P0       | Score trends, query success rate, failure breakdown, cost vs performance monitoring |
| Diagnoser Agent       | P0       | 5 failure type classifications, root-cause recommendations                          |
| Improver Agent        | P0       | 3–5 variant experiment generation; auto-approve and human-approval modes            |
| Rollback Strategy     | P0       | Versioned pipelines, performance checkpoints, auto/manual rollback                  |
| Approval Framework    | P0       | Auto-Approve / Human-in-Loop / Mandatory Gate — risk-based                          |
| Pipeline Configurator | P0       | 5-question no-code setup → YAML config + scoring weights + system prompt            |
| Experiment Tracker    | P0       | Versioned run storage, comparison table, CSV/JSON export                            |
| FastAPI Service       | P0       | 17 REST endpoints, Celery job queue, OpenAPI docs                                   |
| YAML Pipeline Config  | P0       | Declarative, version-controllable pipeline definition                               |
| Docker Compose        | P0       | Single-command local deployment                                                     |

### 5.2 Out of Scope — MVP

| Feature                              | Reason Deferred                                      |
|--------------------------------------|------------------------------------------------------|
| Deployer Agent (K8s full automation) | Requires production infrastructure; post-MVP Phase 3 |
| Custom eval set upload (CSV)         | Synthetic eval sufficient for MVP                    |
| Multi-tenant / user auth             | Not required for self-hosted enterprise MVP          |
| Fine-tuning integration              | Out of scope for RAG improvement loop                |
| GraphQL API                          | REST sufficient for MVP; GraphQL adds complexity     |
| Slack / email alerting               | Deferred to Phase 2 monitoring features              |



## 06 System Architecture

### 6.1 Architecture Layers

| Layer                    | Components                                                                                     |
|--------------------------|------------------------------------------------------------------------------------------------|
| Dashboard (Next.js)      | Score Card   Experiment Tracker   Observer Dashboard   Approval UI   Failure Heatmap   Config  |
| FastAPI Service (Python) | /ingest   /build   /evaluate   /observe   /diagnose   /improve   /approve   /query   /rollback |
| Task Queue               | Celery + Redis — async job execution for all long-running agent operations                     |
| Agents                   | Ingestion   Builder   Evaluator   Observer   Diagnoser   Improver   Deployer                   |
| Data Layer               | PostgreSQL (metadata + audit)   Qdrant (vectors)   MinIO (docs)   MLflow (experiments)         |
| External AI              | Configured LLM provider (generation + judging)   Embedding models (cloud or local)             |

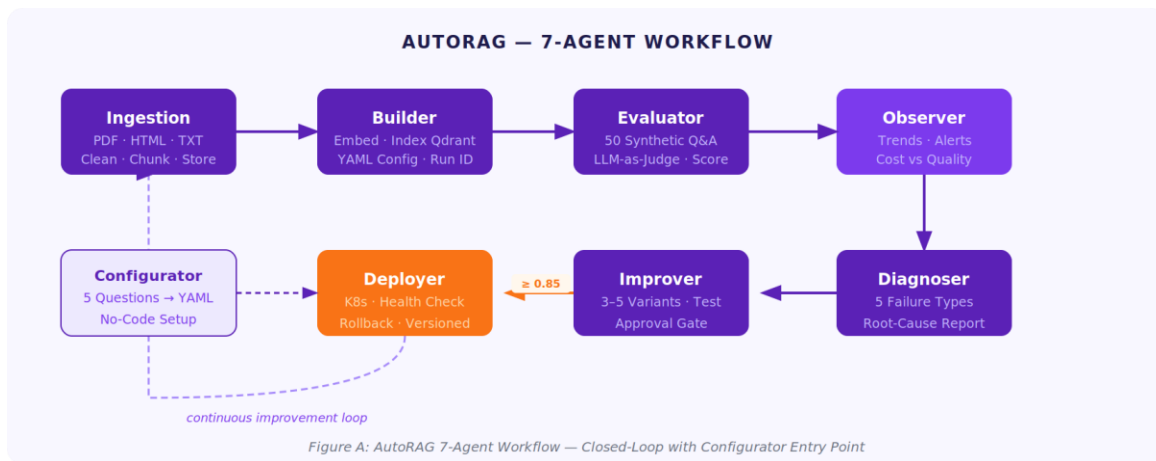


Figure 1: AutoRAG 7-Agent Closed-Loop Workflow with Configurator Entry Point

### 6.2 Data Flow

| Step        | Flow                                                                                      |
|-------------|-------------------------------------------------------------------------------------------|
| 1. Ingest   | Upload → clean → chunk → MinIO (chunks) + PostgreSQL (metadata)                           |
| 2. Build    | Chunks → embed → Qdrant index + run record in PostgreSQL                                  |
| 3. Evaluate | Generate 50 synthetic Q&A → run each through RAG → Unified Score → store in PostgreSQL    |
| 4. Observe  | Collect score trends, query success rates, failure breakdown, cost vs performance metrics |
| 5. Diagnose | Read failures → classify into 5 types → root-cause report → store in PostgreSQL           |
| 6. Approve  | Risk assessment → Auto-Approve / Human Review / Mandatory Gate                            |
| 7. Improve  | Read approval + diagnosis → generate variants → run all → select best by Unified Score    |
| 8. Deploy   | Safe deploy protocol: canary 10% → health check → full promote or rollback                |
| 9. Query    | User question → embed → Qdrant top-k → prompt + LLM → answer + sources                    |

## 07 Agentic AI Design

### 7.1 Agent Definitions

| Agent     | Input                           | Output                                              | LLM Used                           |
|-----------|---------------------------------|-----------------------------------------------------|------------------------------------|
| Ingestion | PDF/HTML/TXT files              | Cleaned chunks in MinIO + metadata in PostgreSQL    | None (rule-based)                  |
| Builder   | Chunks + YAML config            | Qdrant collection + pipeline run record             | None (rule-based)                  |
| Evaluator | Pipeline run ID + chunks        | Evaluation report with Unified Score                | Configured LLM (Q&A gen + judging) |
| Observer  | Evaluation history + query logs | Score trends, success rates, cost-vs-quality graphs | None (analytics)                   |
| Diagnoser | Evaluation report (failures)    | Failure distribution + per-question classification  | Configured LLM (classification)    |
| Improver  | Diagnosis + current config      | Best variant config + experiment table              | None (rule-based variant gen)      |
| Deployer  | Best run ID + deployment config | Running deployment + health status                  | None (rule-based)                  |

### 7.2 5 Failure Types — Diagnoser

| Type              | Definition                                      | Symptom                                                     |
|-------------------|-------------------------------------------------|-------------------------------------------------------------|
| Retrieval Failure | Correct answer not in retrieved chunks          | Low retrieval score; quality OK when context given manually |
| Context Missing   | Answer exists in docs but not retrieved         | Low retrieval recall; chunks don't cover the answer         |
| Hallucination     | Answer contradicts or fabricates beyond context | Faithfulness < 0.5                                          |
| Irrelevant Answer | Answer doesn't address the question             | Quality < 0.4                                               |
| Partial Answer    | Answer is incomplete                            | Quality 0.4–0.6; faithfulness OK                            |

### 7.3 Improver Variant Generation Rules

| Failure Type      | Variants Generated                                  |
|-------------------|-----------------------------------------------------|
| Retrieval Failure | chunk_size=[128,256,512], retriever=[dense, hybrid] |
| Context Missing   | top_k=[5,10,15], reranker=[on,off]                  |
| Hallucination     | temperature=[0.1,0.3], prompt_strictness=[high]     |
| Irrelevant Answer | prompt_template=[precise, conversational]           |
| Partial Answer    | top_k=[10,15], context_window=[2000,4000]           |

### 7.4 Guardrails

| Guardrail                   | Implementation                                                               |
|-----------------------------|------------------------------------------------------------------------------|
| Prompt injection protection | User document text passed as data (not instruction); system prompt hardcoded |
| PII filtering               | Presidio PII scanner on all chunks; PII tokens replaced with [REDACTED]      |
| Output validation           | All LLM outputs validated against Pydantic JSON schemas before storage       |



## 08 Feature Breakdown

### 8.1 Ingestion Layer (P0)

Document loading, cleaning, and chunking pipeline. Supports PDF, HTML, plain text.

- Accepts PDF, HTML, plain text formats
- Fixed-size chunker: configurable chunk\_size (128–2048 tokens) and overlap (0–20%)
- Semantic chunker: splits on sentence boundaries, merges to token targets
- Cleaner removes headers/footers, page numbers, boilerplate
- Each chunk stores: content, source\_doc\_id, chunk\_index, token\_count, strategy
- Ingestion status trackable via API (/ingest/{job\_id}/status)

### 8.2 Builder Agent (P0)

Automated embedding pipeline. Reads YAML config, embeds chunks, indexes into Qdrant.

- Supports text-embedding-3-large (OpenAI) and all-MiniLM-L6-v2 (local)
- All parameters configurable via YAML — no code changes needed
- Embeds in batches (default: 100); retries on rate limit (max 3, exponential backoff)
- Creates named Qdrant collection per run: autorag\_run\_{id}
- Records: run\_id, config\_snapshot, start/end time, chunk\_count, vector\_count

### 8.3 RAG Runtime (P0)

- Dense retrieval: cosine similarity in Qdrant, configurable top\_k (1–20)
- Hybrid retrieval: dense + BM25, configurable alpha blend (0.0–1.0)
- Prompt template: configurable system prompt + context injection
- Returns: answer, source chunks (doc\_id, page, chunk\_index), latency\_ms, token\_count
- p95 response time < 3 seconds for corpora < 10K chunks
- No context found: returns 'I don't have enough information from the provided documents.'

### 8.4 Evaluator Agent (P0)

- Generates N synthetic Q&A pairs (default: 50) using configured LLM
- Q&A includes: question, expected\_answer, source\_paragraph, difficulty\_level (easy/medium/hard)
- Scores: Retrieval (chunk overlap), Quality (LLM judge), Faithfulness (LLM judge), Latency, Cost
- Unified Score:  $0.25 \times \text{Retrieval} + 0.35 \times \text{Quality} + 0.25 \times \text{Faithfulness} - 0.10 \times \text{Latency} - 0.05 \times \text{Cost}$
- All scores normalised to [0.0, 1.0]; evaluation completes in < 5 minutes for 50 Q&A

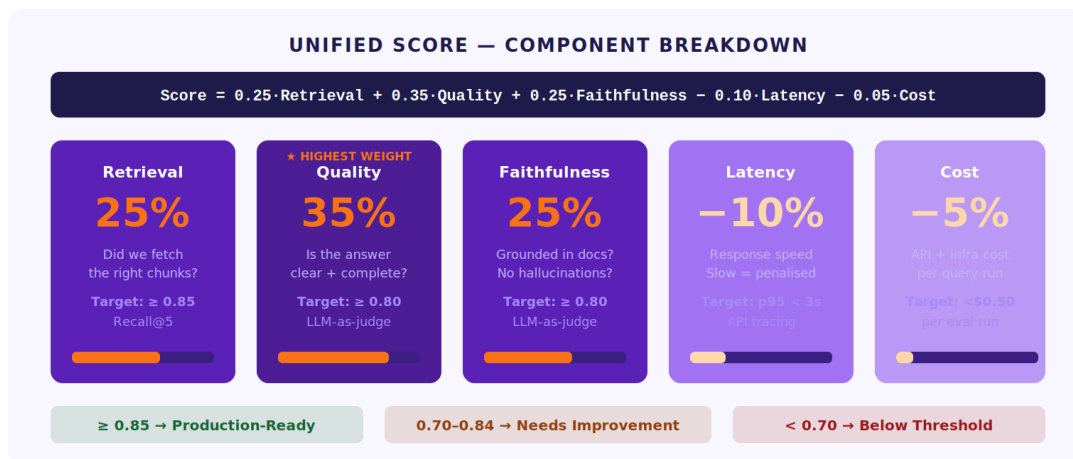


Figure 2: Unified Score Component Breakdown — Weights, Targets, and Thresholds

### 8.5 Diagnoser Agent (P0)

- Classifies every failed question (score < 0.6) into one of 5 failure types
- Shows failure type distribution as percentages
- Per-question view: retrieved chunks, generated answer, expected answer, failure type
- Root-cause recommendation generated per dominant failure type
- Classification accuracy ≥ 90% on test set of 200 known failures

### 8.6 Improver Agent (P0)

- Generates 3–5 variants based on dominant failure type (see Section 7.3)
- Each variant is a full pipeline run: build + evaluate + score
- All variants tracked in Experiment Tracker
- Best variant identified by highest Unified Score
- Approval gate applied before any deployment (see Section 11)
- Shows: 'Best variant improved score from X to Y (+Z)'

### 8.7 Experiment Tracker (P0)

- Stores per run: run\_id, timestamp, config\_snapshot (full YAML), 5 metric scores, status
- Comparison table: select 2+ runs → side-by-side parameter diff + score diff
- Filter: score range, date, config parameter, status
- Export run data as CSV or JSON

### 8.8 FastAPI Service (P0)

- All core operations exposed via REST API (see Section 13 for full endpoint list)
- API key authentication (X-API-Key header); Pydantic request validation
- OpenAPI docs at /docs; long-running ops return job\_id with status polling
- Rate limit: 100 req/min per API key (configurable)

### 8.9 YAML Pipeline Config (P0)

Declarative pipeline configuration — all parameters in one file, version-controllable.

- Configures: chunker, chunk\_size, overlap, embedding\_model, retriever\_type, top\_k, alpha, generator\_model, temperature, system\_prompt, eval.n\_questions, scoring.weights
- Validated against JSON schema on load; invalid config rejected with specific error
- Config stored with each pipeline run for full reproducibility

### 8.10 Dashboard UI (P1)

- Score Card: Unified Score with trend vs previous run
- Score Breakdown: 5-component bar chart with weighted contributions
- Experiment Tracker: filterable, sortable table of all runs
- Observer Dashboard: score trend chart, query success rate, cost vs performance graph
- Improvement Approval: diff view, score delta, approve/reject with audit log
- Failure Heatmap: document sections colour-coded by failure frequency
- Responsive: full layout at 1280px+; tablet at 768px+

### 8.11 Deployer Agent (P1)

- Generates Docker image for selected pipeline run
- Creates Kubernetes Deployment + Service + ConfigMap
- Post-deploy health check: 10 test queries; score ≥ threshold
- Rollback to previous deployment — one-click in dashboard or POST /deploy/rollback

### 8.12 Feedback Loop (P1)

- All production queries logged: question, answer, latency, timestamp
- Thumbs up/down feedback endpoint per answer
- Negative feedback queries added to failure analysis dataset
- New eval run can include production queries as test set

## 09 Observer Agent — Monitoring & Observability

The Observer Agent provides continuous visibility into pipeline health — detecting quality degradation before it impacts users and surfacing cost-vs-quality trade-offs automatically.

### 9.1 Observable Signals

| Signal                | Metric                                | Alert Threshold                               |
|-----------------------|---------------------------------------|-----------------------------------------------|
| Pipeline Score Trends | Unified Score per run over time       | Drop > 0.10 vs last run → alert               |
| Query Success Rate    | % queries scoring $\geq 0.70$         | < 80% success rate → warn                     |
| Failure Breakdown     | Distribution across 5 failure types   | Any type > 40% of failures → alert            |
| Cost vs Performance   | Cost per query vs Unified Score delta | Cost increase > 20% with no score gain → warn |
| API Request Rate      | Requests per minute                   | > 200 req/min → warn                          |
| API Error Rate (5xx)  | Error percentage                      | > 1% → alert                                  |
| API p95 Latency       | Response time histogram               | > 5s → alert                                  |
| LLM API Error Rate    | External API failures                 | > 5% → alert                                  |
| Qdrant Query Latency  | Vector search time                    | > 500ms → warn                                |

### 9.2 AI-Specific Monitoring

| Signal                   | Storage                              | Frequency             |
|--------------------------|--------------------------------------|-----------------------|
| Prompt + response logs   | PostgreSQL llm_audit table           | Every LLM call        |
| Evaluation score history | PostgreSQL evaluations table         | Every eval run        |
| Model cost tracking      | Token counts aggregated by run       | Every eval run        |
| Answer quality drift     | Automated eval on production queries | Weekly (configurable) |

### 9.3 Key Metrics for Alerting

| Metric               | Type      | Alert Threshold                 |
|----------------------|-----------|---------------------------------|
| API request rate     | Counter   | > 200 req/min → warn            |
| API error rate (5xx) | Counter   | > 1% → alert                    |
| API p95 latency      | Histogram | > 5s → alert                    |
| Celery queue depth   | Gauge     | > 50 pending → warn             |
| Eval score decay     | Gauge     | Drop > 0.10 vs last run → alert |

### 9.4 Distributed Tracing

OpenTelemetry SDK across FastAPI service and all agent modules. Trace spans: API request → Celery task → agent execution → LLM call → DB write. Every API request carries X-Trace-ID header.

## 10 Rollback Strategy

Every deployment is versioned. AutoRAG never overwrites a running pipeline — it creates a new version, monitors health, and rolls back automatically if quality degrades.

### 10.1 Versioned Pipeline Architecture

| Component          | Versioning Mechanism                                                         |
|--------------------|------------------------------------------------------------------------------|
| Pipeline Config    | Full YAML snapshot stored per run_id in PostgreSQL JSONB                     |
| Vector Collections | Separate Qdrant collection per run: autorag_run_{id}                         |
| Evaluation Reports | Immutable per-run evaluation record with all 5 metric scores                 |
| Deployment State   | deployments table tracks status, deployed_at, health_check_score per version |

### 10.2 Auto-Rollback Triggers

| Trigger           | Condition                                               | Action                                                 |
|-------------------|---------------------------------------------------------|--------------------------------------------------------|
| Score Degradation | Post-deploy health check score drops > 0.10 vs baseline | Immediate rollback to previous deployment; alert fired |
| Latency Spike     | p95 latency exceeds 5s within 10 min of deploy          | Rollback + latency alert                               |
| Error Rate        | API error rate > 1% within 15 min of deploy             | Rollback + error alert                                 |
| Manual Rollback   | Admin triggers via dashboard or POST /deploy/rollback   | Rollback to any prior versioned deployment             |
| Score Gate        | Improver variant score < current best score             | Block deployment; log result; keep current version     |

### 10.3 Safe Deploy Protocol

- ▶ Step 1: **New pipeline version built and evaluated** — Unified Score calculated
- ▶ Step 2: **Approval gate** — auto-approve if score  $\geq$  threshold + low risk; else human review
- ▶ Step 3: **Canary deploy** — route 10% of traffic to new version for 15 minutes
- ▶ Step 4: **Health check** — run 10 synthetic test queries; verify score  $\geq$  threshold
- ▶ Step 5: **Full promotion or rollback** — based on health check results and Observer alerts
- ▶ Step 6: **Previous version retained for 30 days** as rollback target



## 11 Improvement Approval Framework

AutoRAG supports two improvement modes — fully autonomous (default) and human-in-the-loop (enterprise). Risk-based gating ensures high-impact changes always require human approval.

### 11.1 Approval Modes

| Mode           | Trigger                                                     | Behaviour                                                      | Target User                            |
|----------------|-------------------------------------------------------------|----------------------------------------------------------------|----------------------------------------|
| Auto-Approve   | Score improvement $\geq +0.05$ AND low-risk change          | Variant deployed automatically after safe deploy protocol      | Indie developers, fast-iteration teams |
| Human-in-Loop  | Enterprise mode enabled in config OR medium-risk change     | All improvements require explicit human approval before deploy | Enterprise teams, regulated industries |
| Mandatory Gate | High-impact change detected (model swap, prompt, embedding) | Always requires human approval — non-configurable safety gate  | All users — cannot be overridden       |

### 11.2 Risk Classification

| Change Type                            | Risk Level | Approval Required                          |
|----------------------------------------|------------|--------------------------------------------|
| top_k adjustment                       | Low        | Auto-approve if score improves             |
| chunk_size adjustment                  | Low        | Auto-approve if score improves             |
| temperature adjustment                 | Low        | Auto-approve if score improves             |
| Retrieval type change (dense ↔ hybrid) | Medium     | Auto-approve with 30-min canary monitoring |
| Embedding model change                 | High       | Human approval always required             |
| System prompt change                   | High       | Human approval always required             |
| Generator model change                 | High       | Human approval always required             |

### 11.3 Approval Workflow

- ▶ Improver generates variants and ranks by Unified Score
- ▶ Best variant assessed against risk classification table
- ▶ Low/medium risk + auto-approve mode → safe deploy protocol initiated automatically
- ▶ High risk or human-in-loop mode → approval request surfaced in dashboard with diff view
- ▶ Approver reviews: config diff, score delta, failure type improvement breakdown
- ▶ Approve → safe deploy protocol → monitoring; Reject → log rejection; keep current version
- ▶ All approval decisions logged to audit\_log table (immutable, insert-only)

## IMPROVEMENT APPROVAL FRAMEWORK

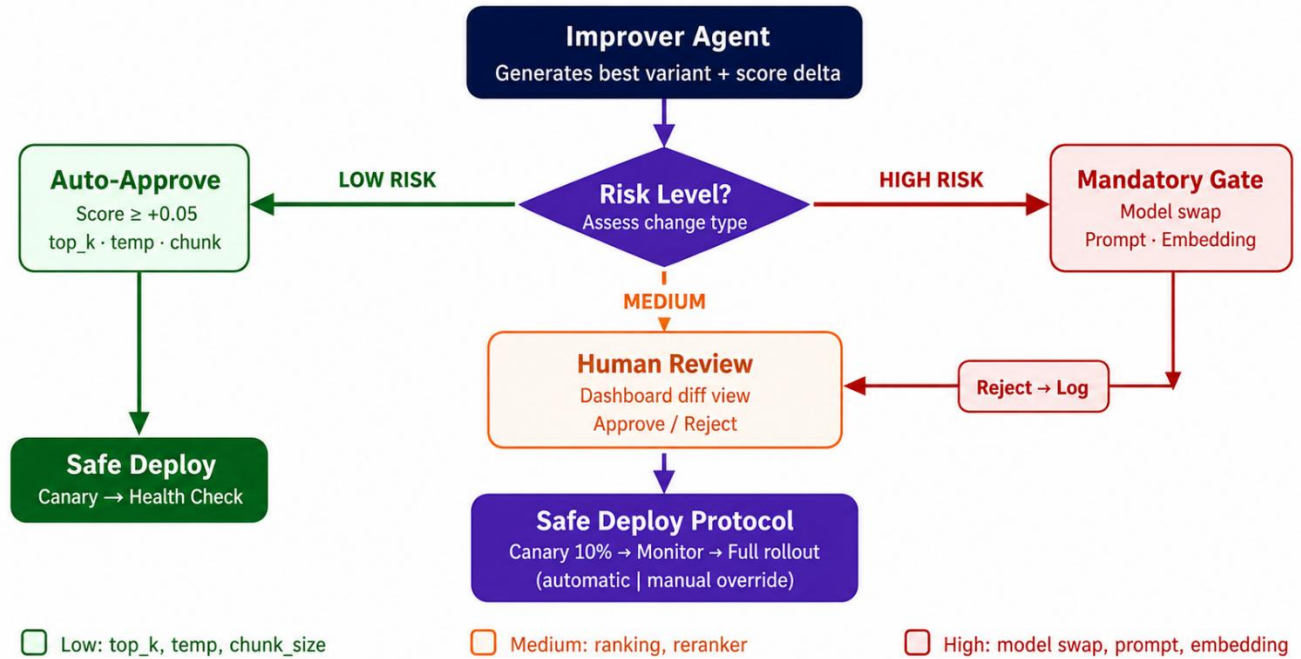


Figure D: AutoRAG Improvement Approval Framework — Risk-Based Decision Tree

Figure 3: Improvement Approval Framework — Risk-Based Decision Tree

## 12 Pipeline Configurator — Feature Spec

The Pipeline Configurator is a guided, no-code setup interface built into AutoRAG. It uses AutoRAG's own configuration engine — not dependent on any specific external AI provider.

### 12.1 User Story

As any user — regardless of technical background — I want to configure a domain-tuned RAG pipeline by answering plain-English questions, so I don't need to understand YAML or pipeline internals.

### 12.2 5-Question Flow & Technical Mapping

| #  | Question                                                                                          | Technical Output                                                                     |
|----|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Q1 | What is your domain? (Legal / Healthcare / Financial / HR / Customer Support / Research / Custom) | Chunking strategy preset, scoring weight profile, system prompt template             |
| Q2 | Who are your end users? (Internal employees / External customers / Developers / Researchers)      | System prompt language style; answer completeness vs brevity weighting               |
| Q3 | What is your biggest concern? (Hallucination / Speed / Cost / Completeness)                       | Scoring weight emphasis — faithfulness, latency, cost, or quality raised accordingly |
| Q4 | How should improvements be applied? (Auto-approve / Human review / Always ask me)                 | approval_mode in YAML config: auto   human   mandatory                               |
| Q5 | Where will this deploy? (Local / Cloud managed / On-premise enterprise)                           | Storage configuration, security defaults, infrastructure sizing recommendations      |

### 12.3 Output Schema

- ▶ YAML config: **Complete pipeline\_config.yaml with all parameters pre-filled**
- ▶ Scoring weight profile: **Custom weights for the 5 Unified Score components based on Q3 answer**
- ▶ System prompt template: **Domain-appropriate prompt pre-written; user edits the [CONTEXT] placeholder**
- ▶ Plain-English summary: **One sentence per decision explaining what was set and why**

### 12.4 Acceptance Criteria

- ▶ 5-question flow completes in under 60 seconds
- ▶ Output YAML is valid against JSON schema and accepted by Builder Agent without modification
- ▶ All output fields are editable — Configurator is a starting point, not a constraint
- ▶ No external AI provider dependency — Configurator uses AutoRAG's rule-based config engine
- ▶ Available in all tiers including Open Source (free)

## 13 API & Integration Requirements

### 13.1 Internal API Endpoints

| Method | Endpoint                       | Description                                                  |
|--------|--------------------------------|--------------------------------------------------------------|
| POST   | /ingest                        | Start ingestion job (upload files or URL list)               |
| GET    | /ingest/{job_id}               | Get ingestion job status                                     |
| POST   | /pipelines                     | Create new pipeline run from YAML config                     |
| GET    | /pipelines                     | List all pipeline runs                                       |
| GET    | /pipelines/{run_id}            | Get pipeline run details + metrics                           |
| POST   | /pipelines/{run_id}/evaluate   | Start evaluation for a pipeline run                          |
| GET    | /pipelines/{run_id}/evaluation | Get evaluation report                                        |
| GET    | /pipelines/{run_id}/observe    | Get Observer Agent metrics and trends                        |
| POST   | /pipelines/{run_id}/diagnose   | Run Diagnoser on evaluation                                  |
| POST   | /pipelines/{run_id}/improve    | Run Improver Agent (returns variants + approval status)      |
| POST   | /pipelines/{run_id}/approve    | Approve best variant for deployment (human-in-loop)          |
| POST   | /configurator                  | Run Pipeline Configurator — 5 questions → YAML config output |
| GET    | /experiments                   | List all experiments with scores                             |
| GET    | /experiments/compare           | Compare 2+ runs (query params: run_ids)                      |
| POST   | /query                         | Query the active pipeline                                    |
| POST   | /deploy/{run_id}               | Deploy pipeline (with safe deploy protocol)                  |
| POST   | /deploy/rollback               | Rollback to previous deployment                              |
| GET    | /jobs/{job_id}                 | Get async job status                                         |

### 13.2 External Integrations

| Service                                  | Purpose                           | Auth                                |
|------------------------------------------|-----------------------------------|-------------------------------------|
| Configured LLM (Claude / OpenAI / local) | Generation + LLM-as-judge scoring | API key (env var) or local endpoint |
| OpenAI Embeddings                        | text-embedding-3-large            | API key (env var)                   |
| HuggingFace (local)                      | all-MiniLM-L6-v2                  | None (local model)                  |

## 14 Data & Storage Design

### 14.1 Storage Architecture

| Data Type                  | Storage                            | Notes                                     |
|----------------------------|------------------------------------|-------------------------------------------|
| Raw documents (PDF/HTML)   | MinIO object store                 | On-premise; no cloud dependency           |
| Document chunks + metadata | PostgreSQL + MinIO                 | Chunk text in PG; binary in MinIO         |
| Document embeddings        | Qdrant (1536-dim or 384-dim)       | One collection per pipeline run           |
| Pipeline run configs       | PostgreSQL (JSONB)                 | Full YAML snapshot per run                |
| Evaluation reports         | PostgreSQL                         | Per-question + aggregate scores           |
| Experiment history         | PostgreSQL + MLflow                | Parameters + metrics versioned            |
| Observer metrics           | PostgreSQL time-series             | Score trends, query logs, cost tracking   |
| Approval decisions         | PostgreSQL audit_log (insert-only) | Immutable — approver, decision, timestamp |
| Production query logs      | PostgreSQL                         | For feedback loop + Observer integration  |

### 14.2 Key Database Entities

| Table          | Key Fields                                                                |
|----------------|---------------------------------------------------------------------------|
| documents      | id, filename, file_type, ingestion_status, storage_path                   |
| chunks         | id, document_id, chunk_index, content, token_count, chunk_strategy        |
| pipeline_runs  | id, name, config JSONB, status, embedding_model, collection_name          |
| evaluations    | id, pipeline_run_id, unified_score, retrieval/quality/faithfulness scores |
| eval_questions | id, evaluation_id, question, answer, failure_type, flagged                |
| diagnoses      | id, pipeline_run_id, failure_distribution JSONB, recommendations          |
| experiments    | id, pipeline_run_id, parent_run_id, is_best_variant, improvement_delta    |
| deployments    | id, pipeline_run_id, status, deployed_at, health_check_score              |
| approval_log   | id, pipeline_run_id, approver, decision, risk_level, timestamp            |
| query_logs     | id, pipeline_run_id, question, answer, latency_ms, user_feedback          |
| audit_log      | id, event_type, user, payload JSONB, timestamp — insert-only              |

### 14.3 Qdrant Vector Store Design

| Parameter         | Value                                                  |
|-------------------|--------------------------------------------------------|
| Collection naming | autorag_{pipeline_run_id}                              |
| Vector dimensions | 1536 (OpenAI) or 384 (MiniLM)                          |
| Distance metric   | Cosine similarity                                      |
| Payload fields    | doc_id, chunk_index, content, source_file, page_number |
| Indexing          | HNSW (Qdrant default)                                  |

### 14.4 Data Privacy

- Presidio PII scanner applied at ingestion; PII tokens replaced with [REDACTED]
- All storage self-hosted by default; no cloud storage dependencies
- Document deletion API cascades to chunks, vectors, and eval data (GDPR-ready)
- API keys and secrets masked in all log outputs

## 15 Non-Functional Requirements

### 15.1 Performance

| Metric                           | Target        |
|----------------------------------|---------------|
| /query p50 latency               | < 1.5 seconds |
| /query p95 latency               | < 3 seconds   |
| /query p99 latency               | < 5 seconds   |
| Pipeline build (1000 chunks) p50 | < 2 minutes   |
| Evaluation (50 questions) p50    | < 5 minutes   |
| Dashboard load time              | < 2 seconds   |
| Experiment comparison table      | < 500ms       |

### 15.2 Security

| Layer              | Mechanism                                                |
|--------------------|----------------------------------------------------------|
| API authentication | Static API key (X-API-Key header); configurable rotation |
| Transport          | TLS 1.3 for all external API calls                       |
| Data at rest       | Qdrant + PostgreSQL + MinIO support encryption at rest   |
| Secret management  | All keys in .env (dev) / Kubernetes Secrets (prod)       |
| PII handling       | Presidio-based redaction in Ingestion Agent              |

### 15.3 Cost Constraints

| Item                                       | Target                                   |
|--------------------------------------------|------------------------------------------|
| LLM API cost per evaluation run (50 Q)     | < \$0.50                                 |
| Embedding cost per 1,000 chunks            | < \$0.02 (cloud) / \$0.00 (local MiniLM) |
| Monthly infra (self-hosted, single server) | < \$50                                   |
| Cloud K8s (3-node cluster)                 | < \$200/month                            |

## 16 AI Safety & Governance

### 16.1 Hallucination Handling

- All answers generated with retrieved context injected into the prompt
- System prompt instructs model to answer ONLY from provided context
- Faithfulness score explicitly measures hallucination; < 0.5 = Hallucination failure, blocked from deployment
- Diagnoser Agent tracks hallucination rates per pipeline run

"Answer ONLY using the provided context. If the answer is not in context, say so."

### 16.2 Bias Mitigation

| Risk                             | Mitigation                                                                        |
|----------------------------------|-----------------------------------------------------------------------------------|
| Synthetic Q&A generation bias    | Generate questions across all doc sections; include easy/medium/hard distribution |
| LLM judge inconsistency          | Run judge 3× on borderline scores (0.4–0.6); average result                       |
| Failure type classification bias | Validated against 200 manually-labelled known failures                            |

### 16.3 Compliance

| Standard       | Status                                                                      |
|----------------|-----------------------------------------------------------------------------|
| GDPR / DPDP    | Document deletion API cascades to all derived data                          |
| Data residency | All storage self-hosted by default; no mandatory cloud dependency           |
| SOC 2          | Not certified (MVP); architecture designed for readiness                    |
| HIPAA          | Not certified; healthcare customers must configure compliant infrastructure |

### 16.4 Audit Logging

All audit events stored in audit\_log PostgreSQL table — immutable (insert-only, no updates).

| Event                         | Logged Fields                                               |
|-------------------------------|-------------------------------------------------------------|
| Document ingested             | user, filename, timestamp, chunk_count                      |
| Pipeline run created          | user, config_snapshot, timestamp                            |
| Evaluation run                | run_id, n_questions, unified_score, timestamp               |
| LLM API call                  | model, prompt_tokens, completion_tokens, latency, timestamp |
| Improvement approved/rejected | run_id, approver, decision, risk_level, timestamp           |
| Pipeline deployed             | run_id, environment, deployed_by, timestamp                 |
| Rollback triggered            | run_id, trigger_type, previous_version, timestamp           |
| Document deleted              | document_id, deleted_by, timestamp                          |



## 17 Testing Strategy

### 17.1 Unit Testing Coverage Targets

| Module                                                 | Coverage Target      |
|--------------------------------------------------------|----------------------|
| Ingestion Agent (chunkers, cleaners)                   | ≥ 90%                |
| Evaluator Agent (scoring formula, judge prompts)       | ≥ 90%                |
| Observer Agent (metric aggregation, alert logic)       | ≥ 90%                |
| Diagnoser Agent (failure classification logic)         | ≥ 90%                |
| Improver Agent (variant generation + approval logic)   | ≥ 85%                |
| Rollback logic (trigger conditions, version selection) | ≥ 90%                |
| Pipeline Configurator (question → YAML mapping)        | ≥ 90%                |
| FastAPI endpoint handlers                              | ≥ 80%                |
| Unified Scoring formula                                | 100% (critical path) |

### 17.2 Integration Tests

- Full pipeline run: Ingest → Build → Evaluate → Observe → Diagnose → Approve → Improve (end-to-end)
- API contract tests: all endpoints tested against OpenAPI schema (schemathesis)
- Real Qdrant instance in CI (Docker); not mocked
- Real PostgreSQL in CI; schema applied via Alembic before tests
- Celery task completion verified with real Redis

### 17.3 Load Testing

| Scenario                     | Target                                      |
|------------------------------|---------------------------------------------|
| /query endpoint              | 50 concurrent users, p95 < 3s, 0 errors     |
| Simultaneous pipeline builds | 5 concurrent builds without race conditions |
| Qdrant search under load     | 100 simultaneous searches, p95 < 500ms      |

## 18 Deployment & DevOps

### 18.1 CI/CD Pipeline (GitHub Actions)

| Stage                 | Action                                                         |
|-----------------------|----------------------------------------------------------------|
| 1 — Lint              | ruff + mypy static analysis                                    |
| 2 — Unit Tests        | pytest; coverage check $\geq 80\%$                             |
| 3 — Integration Tests | Docker Compose environment with real Qdrant, PostgreSQL, Redis |
| 4 — Build Images      | Docker images for API + Dashboard                              |
| 5 — Push              | Container registry (GHCR)                                      |
| 6 — Deploy Staging    | K8s rolling update                                             |
| 7 — Smoke Test        | Automated /query test against staging                          |
| 8 — Manual Gate       | Human approval → deploy to production                          |

### 18.2 Environments

| Environment | Infrastructure       | Purpose                                                   |
|-------------|----------------------|-----------------------------------------------------------|
| Development | Local Docker Compose | Individual developer testing; zero configuration overhead |
| Staging     | Single K8s node      | Integration testing + pre-release validation              |
| Production  | 3-node K8s cluster   | Enterprise customer deployment; HA and horizontal scaling |

## 19 UX & UI Requirements

### 19.1 Design System

| Element           | Choice                                      |
|-------------------|---------------------------------------------|
| Component Library | Tailwind CSS + shadcn/ui                    |
| Primary Color     | #5B21B6 (Deep Purple)                       |
| Accent Colors     | #1E1B4B (Dark Navy), #F97316 (Orange)       |
| Typography        | Inter (body), JetBrains Mono (code/metrics) |
| Charts            | Recharts                                    |
| Accessibility     | WCAG 2.1 Level AA                           |

### 19.2 Screen Inventory (20 Screens)

| #   | Screen                                  | Route                                  |
|-----|-----------------------------------------|----------------------------------------|
| S01 | Upload Documents                        | /documents/upload                      |
| S02 | Pipeline Configurator — 5-question flow | /configurator                          |
| S03 | Pipeline Config (YAML editor)           | /pipelines/new                         |
| S04 | Build Status                            | /pipelines/{id}/build                  |
| S05 | Score Card                              | /pipelines/{id}/score                  |
| S06 | Score Breakdown                         | /pipelines/{id}/score/breakdown        |
| S07 | Observer Dashboard                      | /pipelines/{id}/observe                |
| S08 | Diagnosis Report                        | /pipelines/{id}/diagnosis              |
| S09 | Failure Detail                          | /pipelines/{id}/failures/{question_id} |
| S10 | Failure Heatmap                         | /pipelines/{id}/heatmap                |
| S11 | Experiment Tracker                      | /experiments                           |
| S12 | Side-by-Side Comparison                 | /experiments/compare                   |
| S13 | Improvement Approval                    | /pipelines/{id}/improve/approve        |
| S14 | Rollback Management                     | /deployments                           |
| S15 | Score History Chart                     | /pipelines/{id}/history                |
| S16 | Synthetic Q&A Browser                   | /pipelines/{id}/eval-set               |
| S17 | Weight Configurator                     | /pipelines/{id}/scoring/weights        |
| S18 | Setup Wizard                            | /setup                                 |
| S19 | Monitoring Dashboard                    | /monitoring                            |
| S20 | API Docs                                | /docs                                  |

## 20 Timeline & Milestones

### 20.1 2-Week MVP Delivery Plan

| Day       | Milestone                          | Deliverables                                                            |
|-----------|------------------------------------|-------------------------------------------------------------------------|
| Day 1–2   | Foundation                         | Docker Compose stack; PostgreSQL schema; Qdrant + MinIO connected       |
| Day 3     | Ingestion Agent                    | PDF + HTML loader, fixed-size + semantic chunkers, MinIO storage        |
| Day 4     | Builder Agent                      | Embedding pipeline, Qdrant indexing, YAML config                        |
| Day 5     | RAG Runtime + FastAPI              | /query working end-to-end; /ingest + /build endpoints                   |
| Day 6–7   | Evaluator + Observer Agent         | Synthetic Q&A, LLM-as-judge, Unified Scoring, score trend collection    |
| Day 8     | Diagnoser Agent                    | 5 failure type classification, /diagnose endpoint                       |
| Day 9     | Improver + Configurator + Approval | Variant generation, approval modes, /improve + /approve + /configurator |
| Day 10    | Rollback + Experiment Tracker      | Version management, rollback API, run comparison table                  |
| Day 11–12 | Dashboard MVP                      | Score Card, Observer Dashboard, Experiment Tracker, Approval UI         |
| Day 13    | Testing                            | Unit + integration tests; load test /query                              |
| Day 14    | Polish + Docs                      | docker-compose verified; OpenAPI docs; README Quick Start               |

### 20.2 Post-MVP Roadmap

| Phase   | Timeline  | Features                                                                                       |
|---------|-----------|------------------------------------------------------------------------------------------------|
| Phase 2 | Weeks 3–4 | Full Dashboard (Heatmap, Score History, Rollback); Custom eval set upload; Enhanced Monitoring |
| Phase 3 | Month 2   | Deployer Agent (K8s); Feedback Loop; Multi-model support (Gemini, Mistral)                     |
| Phase 4 | Month 3   | Enterprise: RBAC, audit export, SSO, on-premise K8s installer                                  |
| Phase 5 | Month 4+  | SaaS cloud version; managed Qdrant; team collaboration                                         |

## 21 Business Model & Pricing

| Tier                  | Target                  | Pricing            | Includes                                                                                                 |
|-----------------------|-------------------------|--------------------|----------------------------------------------------------------------------------------------------------|
| Open Source (MIT)     | Indie devs, researchers | Free               | Core pipeline + Configurator; no support SLA                                                             |
| Enterprise On-Premise | Teams of 5–50           | ₹12L – ₹25L / year | Full 7-agent system + Observer + Deployer + Human-in-Loop + Rollback + Configurator. K8s installer. SLA. |
| Enterprise Cloud      | Managed preference      | ₹8L – ₹20L / year  | Managed Qdrant + PostgreSQL + API; 99.9% SLA. Same feature set as On-Premise.                            |
| Professional Services | Custom integrations     | ₹1.5L – ₹5L / eng. | SharePoint/Confluence/Notion connectors. Custom chunkers. Dedicated onboarding engineer.                 |

**Break-even at 8 enterprise customers.** Year 1 target of 10 customers delivers profitability within 3 months of first signed contract.

### 21.1 Revenue Projections

| Year              | Customers              | ARR Estimate    | Primary Driver                               |
|-------------------|------------------------|-----------------|----------------------------------------------|
| Year 1 (post-MVP) | 5–10 enterprise        | ₹60L – ₹1.5 Cr  | Founder-led direct sales; pilot deployments  |
| Year 2            | 20–30 enterprise       | ₹2.5 Cr – ₹5 Cr | OSS inbound + SaaS tier + referrals          |
| Year 3            | 60+ enterprise + cloud | ₹8 Cr – ₹15 Cr  | Channel partnerships + enterprise cloud tier |

<sup>†</sup> Internal projections pending pilot validation. Sources: MarketsandMarkets 2023; Gartner 2024.

## 22 Risks & Assumptions

### 22.1 Technical Risks

| Risk                                     | Probability | Impact | Mitigation                                                |
|------------------------------------------|-------------|--------|-----------------------------------------------------------|
| LLM API latency spikes during evaluation | Medium      | High   | Async Celery queue; configurable timeout; retry logic     |
| Qdrant degradation at > 1M vectors       | Low         | High   | HNSW tuning; horizontal scaling guide                     |
| Synthetic Q&A quality too low            | Medium      | High   | Multi-round generation with diversity checks; flagging UI |
| LLM judge score variance > 0.2           | Medium      | Medium | 3× averaging; calibration test set; prompt versioning     |

### 22.2 AI-Specific Risks

| Risk                               | Probability     | Mitigation                                                       |
|------------------------------------|-----------------|------------------------------------------------------------------|
| Hallucination in generated answers | High (inherent) | Faithfulness scoring; strict context-only prompt; block if < 0.5 |
| Failure type misclassification     | Medium          | 200-example validation; confidence score shown                   |
| Improver generates worse variants  | Medium          | All variants tracked; approval gate before applying              |
| PII leaked in synthetic Q&A        | Low             | PII scan before Q&A generation; redact before LLM call           |

### 22.3 Business Risks

| Risk                                          | Probability | Mitigation                                                                 |
|-----------------------------------------------|-------------|----------------------------------------------------------------------------|
| Enterprise procurement > 6 months             | High        | Target fast-moving teams; build OSS community for inbound                  |
| Competitive product from LangChain/LlamaIndex | Medium      | Deepen Observer + Rollback + Configurator story — full loop differentiator |
| On-premise complexity deters customers        | Medium      | Single docker-compose; video demo; guided Configurator                     |

### 22.4 Assumptions

- ▶ Claude/OpenAI API error rate < **1% at normal load** — sufficient for production LLM-as-judge use
- ▶ Synthetic Q&A generation is **sufficient for meaningful evaluation** without human-labeled data
- ▶ Enterprise customers prefer **self-hosted over cloud**
- ▶ 2-week MVP timeline **achievable with 1–2 experienced developers**
- ▶ The 5-failure-type taxonomy covers **≥ 90% of real-world RAG failure modes**

## 23 Open Questions & Sign-Off

### 23.1 Open Questions

| #  | Question                                                               | Priority | Status |
|----|------------------------------------------------------------------------|----------|--------|
| Q1 | Should MVP include reranking (cross-encoder)?                          | High     | Open   |
| Q2 | Maximum document corpus size for MVP? (100MB? 1GB?)                    | High     | Open   |
| Q3 | OpenAI as alternative LLM, or primary model only for MVP?              | Medium   | Open   |
| Q4 | MLflow required for MVP, or PostgreSQL tracker sufficient?             | Medium   | Open   |
| Q5 | When Improver finds no improvement: auto-accept original or block?     | High     | Open   |
| Q6 | Do enterprise customers require SSO before purchase?                   | Medium   | Open   |
| Q7 | Should Unified Scoring weights be fixed or fully configurable per org? | Medium   | Open   |

### 23.2 Approval & Sign-Off

| Name            | Role               | Signature | Date |
|-----------------|--------------------|-----------|------|
|                 | Lead/Designer      |           |      |
| Sudhanshu Kumar | Technical Reviewer |           |      |